



CSE1500 Web Development - Comprehensive Study Summary

This summary covers all 8 lectures with programming examples aligned with the exam format.^{[1][2][3][4][5][6][7][8]}

Lecture 1: HTTP & Web Architecture

Key Concepts

HTTP is a stateless request-response protocol that enables communication between web clients and servers. The protocol uses TCP/IP for reliable data transmission.^[1]

HTTP Methods

- **GET:** Retrieve data from server (parameters via `req.query` in Node.js)
- **POST:** Send data to server (parameters via `req.body` in Node.js)
- **PUT:** Update existing resources (parameters via `req.params` in Node.js)
- **DELETE:** Remove resources from server

HTTP Status Codes

- **2xx Success:** 200 OK (successful request)
- **3xx Redirection:** 301 Moved Permanently
- **4xx Client Error:** 404 Not Found
- **5xx Server Error:** 502 Bad Gateway

URL Structure

```
<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<fragment>
```

Programming Example: HTTP Request Handling

```
// GET request - parameters from query string
app.get('/flights', (req, res) => {
  const { startDate, endDate } = req.query;
  // Process dates...
});

// POST request - parameters from body
app.post('/flights', (req, res) => {
  const { date } = req.body;
  // Create flight...
});

// PUT request - parameters from URL params
app.put('/flights/:id', (req, res) => {
  const { id } = req.params;
  // Update flight...
});
```

Lecture 2: HTML & CSS

HTML Fundamentals

HTML is a markup language that defines webpage structure using tags. Key elements include forms, tables, lists, and semantic tags.^[4]

DOM (Document Object Model)

The DOM represents HTML as a tree structure where each element is an object with properties, methods, and events.^[4]

HTML Forms

Forms collect user input and require proper validation. Use the `required` attribute for HTML-only validation.

Programming Example: Date Picker Form

```
<form id="dateForm">
  <label for="startDate">Start Date:</label>
  <input id="startDate" name="startDate" type="date" required>

  <label for="endDate">End Date:</label>
  <input id="endDate" name="endDate" type="date" required>

  <button id="submitButton" type="submit">Submit</button>
</form>

<script>
  const startDateInput = document.getElementById('startDate');
  const endDateInput = document.getElementById('endDate');

  // Update endDate min when startDate changes
  startDateInput.addEventListener('change', function() {
    endDateInput.min = startDateInput.value;
  });

  const form = document.getElementById('dateForm');
  form.addEventListener('submit', function(event) {
    event.preventDefault();
    submitForm(startDateInput.value, endDateInput.value);
  });
</script>
```

CSS Key Concepts

CSS styles HTML elements using selectors (element, class, id, pseudo-classes). The box model consists of content, padding, border, and margin.^[4]

Lecture 3: JavaScript

Data Types

JavaScript is dynamically typed and weakly typed, meaning variables can change type and implicit type coercion occurs.^[2]

Primitive Types: Boolean, Number, String, null, undefined

Advanced Types: Arrays (heterogeneous, dynamic), Objects (property-value mappings)

Variables and Scope

- `let`: Block-scoped, cannot be redeclared
- `const`: Block-scoped, cannot be reassigned (but object properties can change)
- `var`: Function-scoped (pre-ES6, avoid in modern code)

Programming Example: Array Manipulation

```
let flights = [
  { id: 1, date: new Date(2026, 0, 15) },
  { id: 2, date: new Date(2026, 0, 20) },
  { id: 3, date: new Date(2026, 0, 25) }
];

// Filter flights by date range
function getFlightsInRange(startDate, endDate) {
  return flights.filter(flight =>
    flight.date >= new Date(startDate) &&
    flight.date <= new Date(endDate)
  );
}

// Add new flight
function createFlight(dateString) {
  const [year, month, day] = dateString.split('-');
  const newFlight = {
    id: flights.length + 1,
    date: new Date(parseInt(year), parseInt(month) - 1, parseInt(day))
  };
  flights.push(newFlight);
  return newFlight;
}
```

DOM Manipulation

```
// Selecting elements
const element = document.getElementById('user-id');
const divs = document.getElementsByTagName('div');
const warnings = document.getElementsByClassName('warning');

// Creating and appending elements
const listItem = document.createElement('li');
const text = document.createTextNode('New Item');
listItem.appendChild(text);
document.getElementById('list').appendChild(listItem);

// Modifying elements
document.getElementById('header').innerHTML = 'Hello World!';
document.getElementById('myImage').src = 'landscape.jpg';
```

Event Handling

JavaScript uses event listeners to respond to user interactions.^[2]

```
const button = document.getElementById('btn');
button.addEventListener('click', function() {
    const inputValue = document.getElementById('input').value;
    // Process input...
});
```

Lecture 4: REST & RESTful APIs

REST Principles

REST (REpresentational State Transfer) is an architectural style for building APIs.^[3]

Key Principles:

1. **Uniform Interface:** Resources identified by URIs, manipulated via HTTP methods
2. **Stateless Communication:** Each request contains all necessary information
3. **Client-Server Architecture:** Clear separation of concerns

4. **Cacheability:** Responses indicate if they can be cached
5. **Layered System:** Support for intermediaries (proxies, gateways)

Resource Identification

URIs should identify resources, not actions:^[3]

- ☒ Good: `/movies/tt123456, /flights`
- ☒ Bad: `/movies/createMovie, /flights/start`

HTTP Methods for CRUD

- **GET:** Read/retrieve resources
- **POST:** Create new resources
- **PUT:** Update entire resource
- **PATCH:** Update partial resource
- **DELETE:** Remove resources

Programming Example: Flight API

```
const express = require('express');
const app = express();
app.use(express.json());

let flights = [];
let nextId = 1;

// GET /flights?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD
app.get('/flights', (req, res) => {
  try {
    const { startDate, endDate } = req.query;

    if (!startDate || !endDate) {
      return res.status(400).json({ error: "Missing required parameters" });
    }

    if (!validDate(startDate) || !validDate(endDate)) {
      return res.status(400).json({ error: "Invalid date format" });
    }
  }
});
```

```

    }

    const [startYear, startMonth, startDay] = startDate.split('-').map(Number);
    const [endYear, endMonth, endDay] = endDate.split('-').map(Number);

    const start = new Date(startYear, startMonth - 1, startDay);
    const end = new Date(endYear, endMonth - 1, endDay);

    const filteredFlights = flights.filter(flight =>
      flight.date >= start && flight.date <= end
    );

    const flightIds = filteredFlights.map(f => f.id);
    res.status(200).json(flightIds);
  } catch (error) {
    res.status(500).json({ error: "Internal server error" });
  }
});

// POST /flights
app.post('/flights', (req, res) => {
  try {
    const { date } = req.body;

    if (!date || !validDate(date)) {
      return res.status(400).json({ error: "Invalid date format" });
    }

    const [year, month, day] = date.split('-').map(Number);
    const flightDate = new Date(year, month - 1, day);

    const newFlight = {
      id: nextId++,
      date: flightDate
    };

    flights.push(newFlight);
    res.status(201).json(newFlight);
  } catch (error) {
    res.status(500).json({ error: "Internal server error" });
  }
});

```

```
    }  
  });  
  
  // Helper function  
  function validateDate(dateString) {  
    const regex = /^\d{4}-\d{2}-\d{2}$/;  
    return regex.test(dateString);  
  }  
}
```

Authentication: JWT (JSON Web Tokens)

JWTs are self-contained tokens that carry authentication information without requiring server-side storage. They offer statelessness and scalability but require secure signing keys and appropriate expiration times.^[3]

Lecture 5: Node.js & Express

Node.js Fundamentals

Node.js is a JavaScript runtime built on Chrome's V8 engine for server-side execution. It's event-driven with non-blocking I/O, using a single-threaded event loop.^[5]

Express.js Framework

Express simplifies server development by providing routing, middleware, and request/response handling.^[5]

Request & Response Objects

Request (req):

- `req.query`: GET parameters
- `req.body`: POST data
- `req.params`: URL parameters (e.g., `/users/:id`)

Response (res):

- `res.send()`: Send response

- `res.json()`: Send JSON response
- `res.status()`: Set status code

Programming Example: Complete Express Server

```
const express = require('express');
const app = express();

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// In-memory data store
let flights = [];
let flightId = 1;

// Helper function to validate date format
function validDate(dateString) {
  return /^\\d{4}-\\d{2}-\\d{2}$/.test(dateString);
}

// GET endpoint with query parameters
app.get('/flights', (req, res) => {
  const { startDate, endDate } = req.query;

  if (!startDate || !endDate) {
    return res.status(400).json({
      error: "Both startDate and endDate are required"
    });
  }

  if (!validDate(startDate) || !validDate(endDate)) {
    return res.status(400).json({
      error: "Dates must be in YYYY-MM-DD format"
    });
  }

  const [sYear, sMonth, sDay] = startDate.split('-').map(Number);
  const [eYear, eMonth, eDay] = endDate.split('-').map(Number);
```

```

    const start = new Date(sYear, sMonth - 1, sDay);
    const end = new Date(eYear, eMonth - 1, eDay);

    const result = flights
      .filter(f => f.date >= start && f.date <= end)
      .map(f => f.id);

    res.status(200).json(result);
  });

// POST endpoint with body parameters
app.post('/flights', (req, res) => {
  const { date } = req.body;

  if (!date) {
    return res.status(400).json({ error: "Date is required" });
  }

  if (!validDate(date)) {
    return res.status(400).json({
      error: "Date must be in YYYY-MM-DD format"
    });
  }

  const [year, month, day] = date.split('-').map(Number);

  const newFlight = {
    id: flightId++,
    date: new Date(year, month - 1, day)
  };

  flights.push(newFlight);
  res.status(201).json(newFlight);
});

// PUT endpoint with URL parameters
app.put('/flights/:id', (req, res) => {
  const { id } = req.params;
  const { date } = req.body;

```

```

    const flightIndex = flights.findIndex(f => f.id === parseInt(id));

    if (flightIndex === -1) {
        return res.status(404).json({ error: "Flight not found" });
    }

    if (!validDate(date)) {
        return res.status(400).json({
            error: "Date must be in YYYY-MM-DD format"
        });
    }

    const [year, month, day] = date.split('-').map(Number);
    flights[flightIndex].date = new Date(year, month - 1, day);

    res.status(200).json(flights[flightIndex]);
});

// DELETE endpoint
app.delete('/flights/:id', (req, res) => {
    const { id } = req.params;
    const flightIndex = flights.findIndex(f => f.id === parseInt(id));

    if (flightIndex === -1) {
        return res.status(404).json({ error: "Flight not found" });
    }

    flights.splice(flightIndex, 1);
    res.status(204).send();
});

// Error handling middleware
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({ error: "Something went wrong" });
});

const PORT = 3000;
app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
});

```

```
});
```

Middleware

Middleware functions execute in sequence and can modify request/response objects. They must call `next()` to pass control to the next middleware.^[5]

```
// Custom middleware example
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};

app.use(logger);
```

Lecture 6: Node.js & PostgreSQL

node-postgres (pg)

The `pg` library enables Node.js applications to interface with PostgreSQL databases. It uses connection pooling for efficient query execution.^[7]

Programming Example: Database Integration

```
const { Pool } = require('pg');
const express = require('express');
const app = express();

app.use(express.json());

// Database connection pool
const pool = new Pool({
  user: 'username',
  host: 'localhost',
  database: 'flights_db',
  password: 'password',
  port: 5432,
```

```

});

// GET flights with date range filter
app.get('/flights', async (req, res) => {
  try {
    const { startDate, endDate } = req.query;

    if (!startDate || !endDate) {
      return res.status(400).json({
        error: "Both startDate and endDate required"
      });
    }

    const query = `
      SELECT id FROM flights
      WHERE date >= $1 AND date <= $2
      ORDER BY date ASC
    `;

    const result = await pool.query(query, [startDate, endDate]);
    const flightIds = result.rows.map(row => row.id);

    res.status(200).json(flightIds);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: "Database error" });
  }
});

// POST new flight
app.post('/flights', async (req, res) => {
  try {
    const { date } = req.body;

    if (!validDate(date)) {
      return res.status(400).json({ error: "Invalid date format" });
    }

    const query = 'INSERT INTO flights (date) VALUES ($1) RETURNING *';
    const result = await pool.query(query, [date]);
  }
});

```

```

        res.status(201).json(result.rows[0]);
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Database error" });
    }
});

// GET single flight by ID
app.get('/flights/:id', async (req, res) => {
    try {
        const flightId = parseInt(req.params.id);

        const query = 'SELECT * FROM flights WHERE id = $1';
        const result = await pool.query(query, [flightId]);

        if (result.rows.length === 0) {
            return res.status(404).json({ error: "Flight not found" });
        }

        res.status(200).json(result.rows[0]);
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Database error" });
    }
});

function validDate(dateString) {
    return /^d{4}-d{2}-d{2}$/.test(dateString);
}

```

Key Points:

- Use parameterized queries (\$1, \$2) to prevent SQL injection^[7]
- Access URL parameters via `req.params` (e.g., `req.params.id`)
- Use `async/await` for cleaner asynchronous code
- Always handle errors with try-catch blocks

Lecture 7: Design Patterns & React

Design Patterns

Design patterns are reusable solutions to common software design problems.^[6]

Creational: Singleton, Factory Method

Structural: Adapter, Decorator, Composite

Behavioral: Command, Observer, Strategy

MVC Pattern

Model-View-Controller separates application logic:

- **Model:** Data and business logic
- **View:** User interface
- **Controller:** Handles user input and updates model/view

React Fundamentals

React is a component-based library for building user interfaces. Components are JavaScript functions that return JSX (JavaScript XML).^[6]

Programming Example: React Component

```
// Functional component with state
import React, { useState, useEffect } from 'react';

function FlightList() {
  const [flights, setFlights] = useState([]);
  const [startDate, setStartDate] = useState('');
  const [endDate, setEndDate] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const fetchFlights = async () => {
    if (!startDate || !endDate) return;
  }
}
```

```

    setLoading(true);
    setError(null);

    try {
      const response = await fetch(
        `/flights?startDate=${startDate}&endDate=${endDate}`
      );

      if (!response.ok) {
        throw new Error('Failed to fetch flights');
      }

      const data = await response.json();
      setFlights(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    fetchFlights();
  };

  return (
    <div className="flight-list">
      <form onSubmit={handleSubmit}>
        <label>
          Start Date:
          <input
            type="date"
            value={startDate}
            onChange={(e) => setStartDate(e.target.value)}
            required
          />
        </label>

        <label>

```

```

        End Date:
        <input
            type="date"
            value={endDate}
            min={startDate}
            onChange={(e) => setEndDate(e.target.value)}
            required
        />
    </label>

    <button type="submit">Search Flights</button>
</form>

{loading && <p>Loading...</p>}
{error && <p className="error">{error}</p>}

{flights.length > 0 && (
    <ul>
        {flights.map(id => (
            <li key={id}>Flight ID: {id}</li>
        ))}
    </ul>
)}
</div>
);
}

export default FlightList;

```

React Key Concepts:

- **State:** `useState()` hook manages component data
- **Effects:** `useEffect()` hook handles side effects
- **Props:** Pass data from parent to child components
- **Events:** Handle using camelCase (e.g., `onClick`)
- **Conditional Rendering:** Use `&&` or ternary operators

Lecture 8: WebSockets

Why WebSockets?

WebSockets enable full-duplex, bi-directional communication between client and server over a single persistent connection. Unlike HTTP's request-response model, WebSockets allow real-time data push.^[8]

WebSocket Protocol

WebSockets start with an HTTP upgrade handshake, then maintain an open connection.^[8]

Key Events:

- `open`: Connection established
- `message`: Data received
- `error`: Connection error
- `close`: Connection closed

Programming Example: WebSocket Server & Client

```
// Server (Node.js with ws library)
const express = require('express');
const WebSocket = require('ws');

const app = express();
const server = app.listen(3000);

// WebSocket server on port 8080
const wss = new WebSocket.Server({ port: 8080 });

// Store connected clients
const clients = new Set();

wss.on('connection', (ws) => {
  console.log('New client connected');
  clients.add(ws);

  // Send welcome message
  ws.send(JSON.stringify({
```

```

        type: 'welcome',
        message: 'Connected to flight updates'
    }));

// Handle incoming messages
ws.on('message', (data) => {
    try {
        const message = JSON.parse(data);
        console.log('Received:', message);

        // Broadcast to all clients
        clients.forEach(client => {
            if (client.readyState === WebSocket.OPEN) {
                client.send(JSON.stringify({
                    type: 'update',
                    data: message
                }));
            }
        });
    } catch (error) {
        console.error('Invalid message:', error);
    }
});

// Handle disconnection
ws.on('close', () => {
    console.log('Client disconnected');
    clients.delete(ws);
});

// Handle errors
ws.on('error', (error) => {
    console.error('WebSocket error:', error);
});
});

// Simulate flight updates every 5 seconds
setInterval(() => {
    const update = {
        type: 'flight-update',

```

```

        flight: {
            id: Math.floor(Math.random() * 1000),
            status: 'On Time',
            timestamp: new Date().toISOString()
        }
    };

    clients.forEach(client => {
        if (client.readyState === WebSocket.OPEN) {
            client.send(JSON.stringify(update));
        }
    });
}, 5000);

// Client (Browser)
const socket = new WebSocket('ws://localhost:8080');

socket.addEventListener('open', () => {
    console.log('Connected to WebSocket server');

    // Send initial message
    socket.send(JSON.stringify({
        type: 'subscribe',
        topic: 'flights'
    }));
});

socket.addEventListener('message', (event) => {
    const data = JSON.parse(event.data);
    console.log('Received:', data);

    // Update UI based on message type
    if (data.type === 'flight-update') {
        updateFlightDisplay(data.flight);
    }
});

socket.addEventListener('error', (error) => {
    console.error('WebSocket error:', error);
});

```

```

});

socket.addEventListener('close', () => {
    console.log('Disconnected from server');
});

function updateFlightDisplay(flight) {
    const flightList = document.getElementById('flight-list');
    const listItem = document.createElement('li');
    listItem.textContent = `Flight ${flight.id}: ${flight.status} at
${flight.timestamp}`;
    flightList.appendChild(listItem);
}

```

WebSocket Advantages:^[8]

- Reduced latency (no repeated handshakes)
- Real-time bidirectional communication
- Lower overhead per message

WebSocket Disadvantages:^[8]

- Resource intensive (persistent connections)
- Complex load balancing (stateful connections)
- Security considerations (DoS attacks, hijacking)

Exam-Style Complete Examples

Example 1: Date Picker with Validation

```

<!DOCTYPE html>
<html>
<head>
    <title>Booking Form</title>
</head>
<body>
    <form id="dateForm">

```

```
<label for="startDate">Start Date:</label>
<input id="startDate" name="startDate" type="date" required>

<label for="endDate">End Date:</label>
<input id="endDate" name="endDate" type="date" required>

<button id="submitButton" type="submit">Submit</button>
</form>

<script>
  const startDateInput = document.getElementById('startDate');
  const endDateInput = document.getElementById('endDate');
  const form = document.getElementById('dateForm');

  // Ensure endDate is never before startDate
  startDateInput.addEventListener('change', function() {
    endDateInput.min = startDateInput.value;

    // If endDate is now invalid, clear it
    if (endDateInput.value && endDateInput.value < startDateInput.value) {
      endDateInput.value = '';
    }
  });

  // Handle form submission
  form.addEventListener('submit', function(event) {
    event.preventDefault();

    const start = startDateInput.value;
    const end = endDateInput.value;

    // Both fields required (handled by HTML required attribute)
    // endDate >= startDate (handled by min attribute)

    submitForm(start, end);
  });

  function submitForm(startDate, endDate) {
    console.log('Submitting:', startDate, endDate);
    // Your submission logic here
  }
</script>
```

```
    }  
  </script>  
</body>  
</html>
```

Example 2: Complete Flight API Implementation

```
const express = require('express');  
const app = express();  
  
app.use(express.json());  
  
// In-memory storage  
let flights = [];  
let nextId = 1;  
  
// Validation helper  
function validateDate(dateString) {  
  if (typeof dateString !== 'string') return false;  
  const regex = /^\\d{4}-\\d{2}-\\d{2}$/;  
  if (!regex.test(dateString)) return false;  
  
  const date = new Date(dateString);  
  return date instanceof Date && !isNaN(date);  
}  
  
// GET /flights?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD  
app.get('/flights', (req, res) => {  
  try {  
    const { startDate, endDate } = req.query;  
  
    // Validate parameters exist  
    if (!startDate || !endDate) {  
      return res.status(400).json({  
        error: "Both startDate and endDate are required"  
      });  
    }  
  
    // Validate date format
```

```

    if (!validDate(startDate) || !validDate(endDate)) {
      return res.status(400).json({
        error: "Dates must be in YYYY-MM-DD format"
      });
    }

    // Parse dates (month is 0-indexed in JavaScript Date)
    const [startYear, startMonth, startDay] = startDate.split('-').map(Number);
    const [endYear, endMonth, endDay] = endDate.split('-').map(Number);

    const start = new Date(startYear, startMonth - 1, startDay);
    const end = new Date(endYear, endMonth - 1, endDay);

    // Filter flights (inclusive of both dates)
    const matchingFlights = flights.filter(flight => {
      return flight.date >= start && flight.date <= end;
    });

    // Return only IDs
    const flightIds = matchingFlights.map(f => f.id);
    res.status(200).json(flightIds);

  } catch (error) {
    console.error('Error in GET /flights:', error);
    res.status(500).json({ error: "Internal server error" });
  }
});

// POST /flights
app.post('/flights', (req, res) => {
  try {
    const { date } = req.body;

    // Validate date exists
    if (!date) {
      return res.status(400).json({ error: "Date is required" });
    }

    // Validate date format
    if (!validDate(date)) {

```

```

        return res.status(400).json({
            error: "Date must be in YYYY-MM-DD format"
        });
    }

    // Parse date string to Date object
    const [year, month, day] = date.split('-').map(Number);
    const flightDate = new Date(year, month - 1, day);

    // Create new flight
    const newFlight = {
        id: nextId++,
        date: flightDate
    };

    flights.push(newFlight);

    res.status(201).json(newFlight);

} catch (error) {
    console.error('Error in POST /flights:', error);
    res.status(500).json({ error: "Internal server error" });
}
});

// Error handling middleware
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({ error: "Something went wrong" });
});

const PORT = 3000;
app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
});

module.exports = app; // For testing

```

Key Takeaways for Exam

Request Parameter Access in Node.js

```
// GET - Query parameters
app.get('/resource', (req, res) => {
  const { param1, param2 } = req.query;
});

// POST - Body parameters
app.post('/resource', (req, res) => {
  const { param1, param2 } = req.body;
});

// PUT/DELETE - URL parameters
app.put('/resource/:id', (req, res) => {
  const { id } = req.params;
});
```

Date Handling in JavaScript

```
// IMPORTANT: Month is 0-indexed!
const date = new Date(2026, 0, 15); // January 15, 2026

// From YYYY-MM-DD string
const [year, month, day] = "2026-01-15".split('-').map(Number);
const dateObj = new Date(year, month - 1, day);

// Comparison
date1 >= date2 // Works with Date objects
```

HTML Form Validation

- Use `required` attribute for mandatory fields
- Use `min` and `max` attributes for date inputs
- Use `change` event listener to dynamically update constraints
- Use `event.preventDefault()` to handle form submission in JavaScript

Error Handling Patterns

Always return JSON errors with descriptive messages:

```
if (error) {  
    return res.status(400).json({ error: "Description of error" });  
}
```

This comprehensive summary covers all 8 lectures with practical programming examples aligned with your exam format.^{[1][2][6][7][3][4][5][8]}

**

1. CSE1500_Lecture_1_Intro_HTTP-Copy.pdf
2. CSE1500_Lecture_3_JS-Copy.pdf
3. CSE1500_Lecture_4_REST-Copy.pdf
4. CSE1500_Lecture_2_HTML_CSS-Copy.pdf
5. CSE1500_Lecture_5_Summary-1.pdf
6. CSE1500_Lecture_7_Patterns_React-Copy.pdf
7. CSE1500_Lecture_6_Node_Postgres-Copy.pdf
8. CSE1500_Lecture_8_Web_Sockets-Copy.pdf